Additive manufacturing path planning for a six degrees of freedom open source robot arm

Barriga, J. and Alonso, F.

Abstract Traditional 3D printing which builds a model layer by layer has limitations when it comes to building organic and curved surfaces, this paper will present a brief review of state of the art technologies used for 3D printing using multi axis robots, then a proposal of an inverted mounting frame will be presented for the MOVEO open source robot and then a description of the mathematical modeling and programming of trajectories for the robots. The purpose of this study is to give a solution for 3D printing of complex surfaces with a focus on Bioprinting.

***Index Terms*—3D printing, Additive manufacturing, Multi-axis printing, Robot, Bioprinting, open-source**

I. Introduction

Additive (AM) describes a group of processes that produce objects by depositing materials in a layer-by-layer way . This technology, which is in its infancy, emerged in the late 1980s under the name of rapid prototyping, that was because the main goal of this technology was to build models for design review reducing costs and time of fabrication.. Materials were limited to few polymers, ceramics and metals but gradually new materials were introduced and different technologies of additive manufacturing.. Nowadays the range of materials goes from a wide number of thermoplastics, resins, bio gel, ceramics, concrete.

The main additive manufacturing technology used these days is Fused Deposition Modeling (FDM), in which a filament of thermoplastic is melted layer by layer. This is the cheapest technology and one of the fastest, but it is very limited with complex shapes with curved surfaces.

In recent years the trending has seen an increment in additive manufacturing using Dantry robots, but there are advancements in using robotic arms that allows additive manufacturing in 3D surfaces, as the tool can be placed tangent to the normal of the surface to insert the material directly to the mold. Using a six degrees of freedom robotic arm opens a world of opportunities for manufacturing to evolve, changing the dynamic of the deposition of material thus modifying the mechanical properties of the final piece.

Mathematical modeling of curves using parametric curves for the trajectory of the robot arm allows to plan the algorithm specifically to the kinetic properties of our robot and a specifications. This makes possible to work on a certain position of the tool for custom deposition in the surface directly from almost every position.

The use of parametric curves in robotics go to the path planning of the program, and the more used curves are: Béziers, B-splines, rational Bézier curves(RBCs), and non-uniform rational B-splines(NURBs).

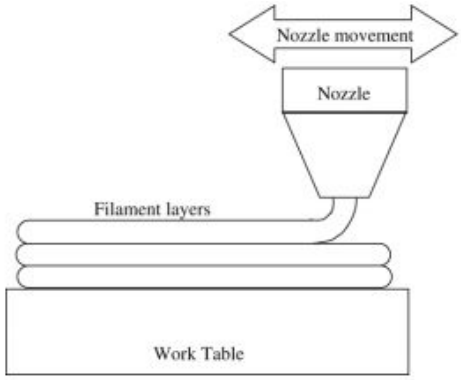
In short, our work makes the following contributions:

- A proposal for an inverted position robot mounting frame
- Mathematical modeling of a path planning for a 6DoF robot
- A simulation of the trajectories.

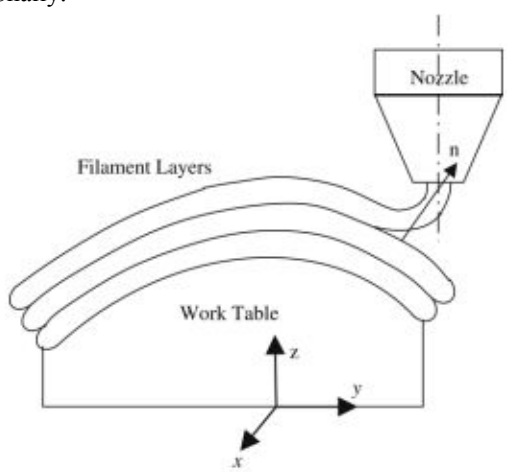
II. Related work

A. Extruder path generation for CLFDM

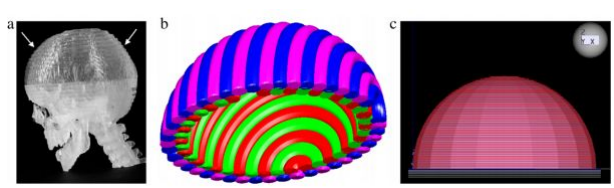
One of the main disadvantages of traditional FDM, is the inability to create smooth curved surfaces without the need of supports.



This paper proposes a new term, Curved layer fused deposition modelling (CLFDM), giving a new building paradigm for FDM in which the material will be deposited along curved paths instead of horizontal paths as used traditionally.



With this comes a limitation, and it is that traditional FDM uses 3 axis CNC that means that in curved surfaces the nozzle can´t be in a normal position with the surface. Ideally a 5 axis CNC will work adjusting this angle so that the extruder will coincide with the normal of the surface.

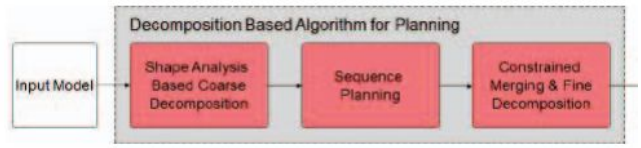


B. RobotFDM: A Robot System for Support-Free Fabrication using FDM

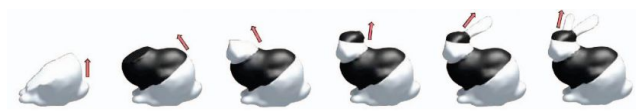
Another approach for additive manufacturing of complex shapes is to split the model in multiple parts to change the direction i order to print in different angles without supports.

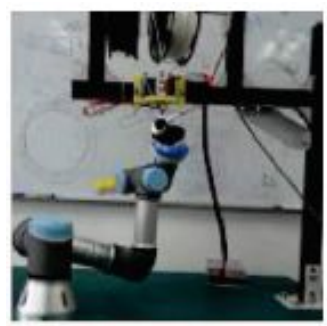
In this paper [3] it's described a system with a static nozzle and a 6 DOF robot arm with a build plate as a tool, this helps moving the model in different angles to accomplish the manufacture.

The Decomposition Based Algorithm for Planning described in the study works in the way shown in the Figure **X**



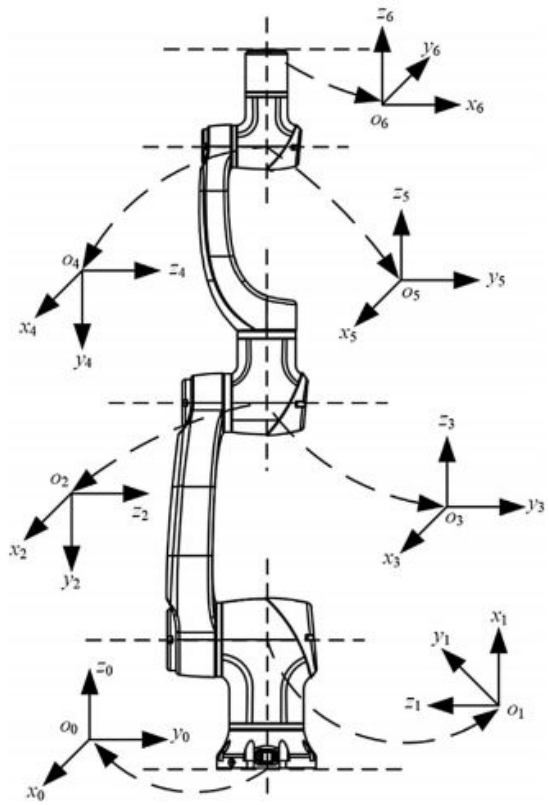
In the study the algorithm was performed in a bunny model, as shown in the Figure **X** it is shown in different colors the multiple parts decomposed. An in the Figure **X** you can see the system with the 6DOF robot and the nozzle mounted on top.





C. *Application of robotic-assisted in situ 3D printing*

This work [1] aimed to report robotic-assisted in situ 3D bio-printing technology for cartilage regeneration and explore its potential in clinical application.



The system consisted on a six degree of freedom (6DOF) robot, an extrusion-based bioprinting unit, an air compressor, a control cabinet and a workbench. An off-line programming software was developed for this robot and the workspace was completed by the *Monte Carlo method* on MaTLAB software.

III. Inverted mounting structure proposal

A. Advantage of Inverted Mounting

The main reason to choose inverted Mounting while installing a Robot, is the increase of reach. Whit a robot mounted traditionally, the reach is limited by the positive or negative of X and Y (relative to robot base). But by inverting the position upside-down you increase the reach by allowing the robot to work in the negative and positive coordinates of X an Y.

B. Parameters considered in the design

For the design´s proposal 2 main parameters where considered: the weight of the robot and the reach.

In this study the total mass considered for the robot itself and the electronics was approximately of 14kg, as the purpose of this robot is to perform bioprinting, 2 more kilograms where considered taking in consideration the syringe tool and the deposit of the material to use.

As for the reach, the maximum reach is 575 mm, in all (+/-) X and Y axis.

C. Materials and Fabrication

The frame is intended to be build with Low-Carbon Steel Square Tubes of 2 inches of width and 0.120 inches of wall thickness. It consists on 10 tubes of 130mm of length welded together as shown in the **Figure X.**

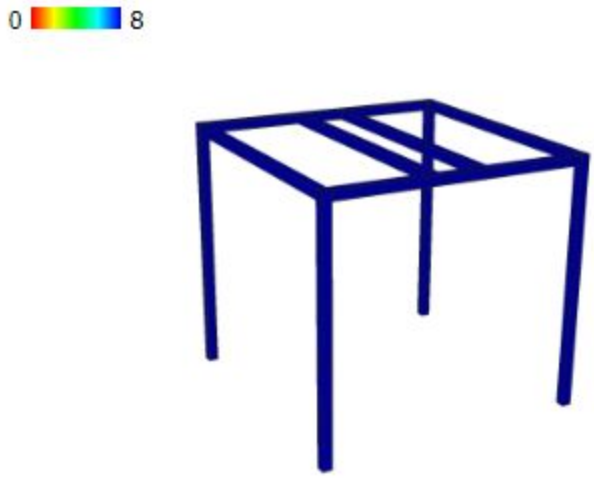
For the mounting plate, a sheet of aluminum 16mm tick was used, with 8 bolts os ¼ ¨- 20 Grade 5 to secure it to the frame.

D. Static Stress Study

The Static Stress was performed only on the frame, as it is the part that will support all the robot and electronics.

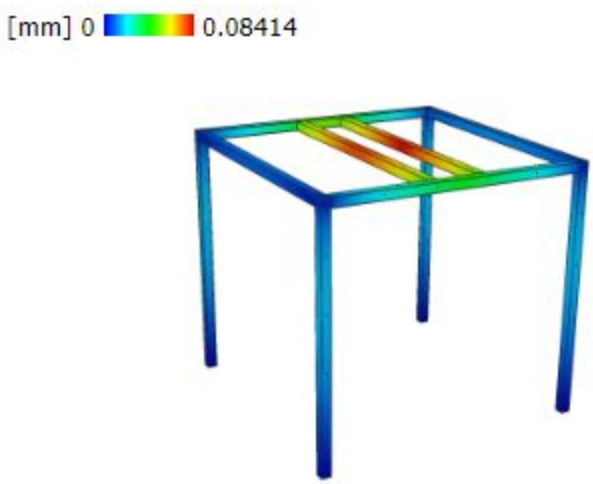
The forces applied were in the center tubes as this will be the ones with the mounting holes for the bolts.

Safety Factor



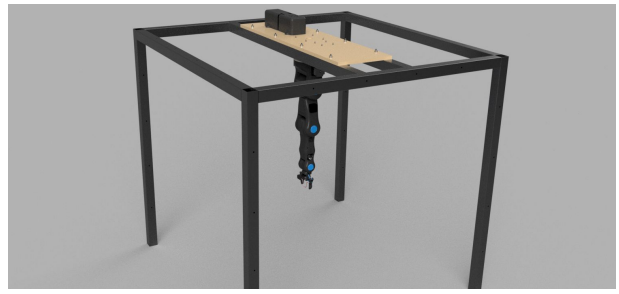
Stress

Displacement



A0

View with Robot Mounted



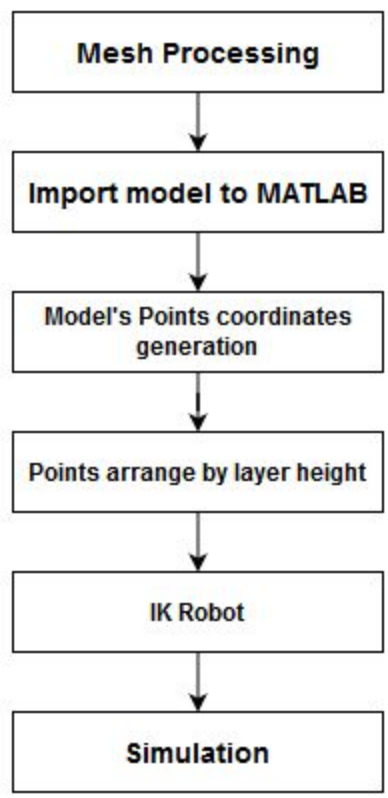
IV. MATHEMATHICAL MODELING THEORY

If you are using *Word,* use either the Microsoft Equation

V. PROGRAMMING THEORY

A. MATLAB Method

The steps towards path generation in MATLAB is the following:



a) Mesh processing, this step is the same shown in the MATLAB method, first you need to clean or generate the model using meshmixer, the main objective is to get a clean smooth surface in STL format so that our program in grasshopper can process it and generate the corresponding path.
b) To import the model into MATLAB we use the command " stlread" , which give us 2 matrices,

one with the coordinates of each point in the model, and the second matrix is the identificator of each triangle in the model, identifying the points within a triangle.

```
STL = stlread('media_esfera.stl');
```

c) Then, in order to manipulate the points on our model, we need to store the values in another variable, vertexID for the triangles and vertexXYZ for the coordinates, and round the values in the z coordinate.

```
vertexID = STL.ConnectivityList;
vertexXYZ = STL.Points;
vertexXYZ(:,3) =
round(vertexXYZ(:,3))
```

d) In order to plot the points in 2D, we need to separate the points by the layer height set at the beginning. Then we need to plot each point by layer.

```
cx=[];
cy=[];
cz=[];
time=0.01;
layer_height = 1;
for layer =
0:layer_height:max(vertexXYZ(:,3))
      for c = 1:size(vertexXYZ,1)
      if vertexXYZ(c,3)== layer

plot3(vertexXYZ(c,1),vertexXYZ(c,2)
,vertexXYZ(c,3),'o')
            hold on
            cx= [cx,
vertexXYZ(c,1)];
            cy= [cy,
vertexXYZ(c,2)];
            cz= [cz,
vertexXYZ(c,3)];
            pause(time)
      end
      %pause(time)
      end
      title(sprintf('Slice Z =
%4.2f', layer))
      pause(time)
end
```

e) For the simulation of the robot in Matlab we use the tools of the Robotics System Toolbox, this allows us to use the functions of RigidBodyTree. With this we can use the Denavit-Hartenberg equations for the development of the robot. This function are in the Toolbox that we used and this simplify the process for creating the simulated robot.

```
InverseKinematics.m
dhparams = [0        0       1       0;
            0        pi/2    0       0
            1         0      0       0;
            0        pi/2    1       0;
            0        -pi/2   0       0;
            0        pi/2    1       0];

robot = rigidBodyTree;                          %robot a usar

body1 = rigidBody('body1');                     %base del robot
jnt1 = rigidBodyJoint('jnt1','revolute');

setFixedTransform(jnt1,dhparams(1,:),'dh');
body1.Joint = jnt1;

addBody(robot,body1,'base')                     %base del robot
```

With this functions we can also use the established configuration for calculating the inverse kinematics for the TCP. This combined with the parameter of our robot and the points in space that will be the configuration desired for the TCP. This points will be originated from Rhino and delivered to Matlab.

```
showdetails(robot)
%show(robot);
axis([-0.5,0.5,-0.5,0.5,-0.5,0.5])
axis on

randConfig = robot.randomConfiguration;         %mueve la base del robot de manera random
tform = getTransform(robot,randConfig,'body6','base');

show(robot,randConfig)

ik = inverseKinematics('RigidBodyTree',robot);  %inverse kinematic para posicion inicial del robo
weights = [0.25 0.25 0.25 1 1 1];
initialguess = robot.homeConfiguration;

[configSoln,solnInfo] = ik('body6',tform,weights,initialguess);
```

f) This are the parameter we find for the Moveo BCN3D, this are used in the function dhparams, and create a simulation of the physical robot.

```
dhparams = [0        0       1       0;
            0        pi/2    0       0
            1         0      0       0;
            0        pi/2    1       0;
            0        -pi/2   0       0;
            0        pi/2    1       0];
```

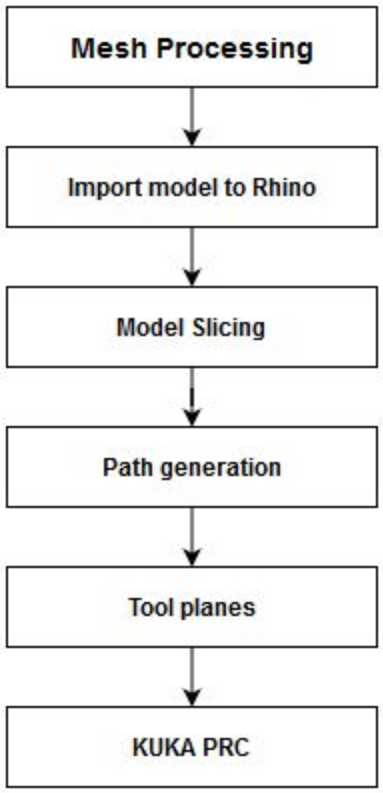
The IK function is used to calculate the solution of the configuration of the joints, then the configSoln, solnInfo will position the robot in the right position.

```
ik = inverseKinematics('RigidBodyTree',robot);  %inverse kinematic pa
weights = [0.25 0.25 0.25 1 1 1];
initialguess = robot.homeConfiguration;

[configSoln,solnInfo] = ik('body6',tform,weights,initialguess);
```

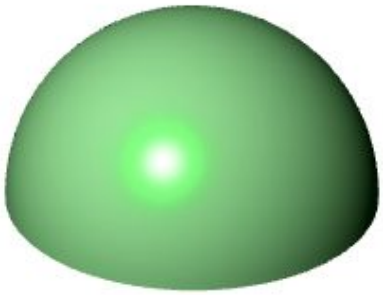
B. Rhino Grasshopper Method

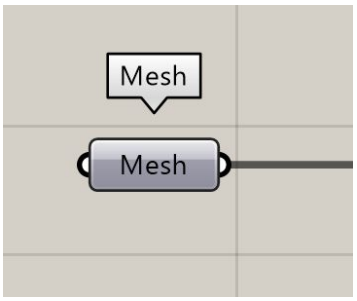
The workflow in Rhino Grasshopper is the following:



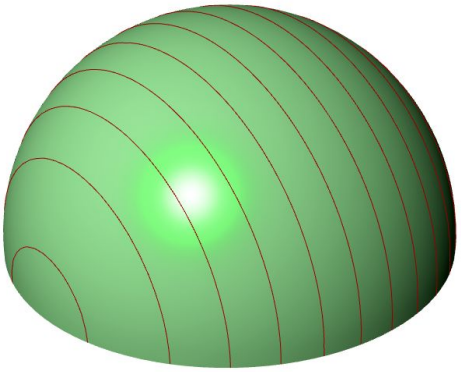
g) Mesh processing, this step is the same shown in the MATLAB method, first you need to clean or generate the model using meshmixer, the main objective is to get a clean smooth surface in STL format so that our program in grasshopper can process it and generate the corresponding path.

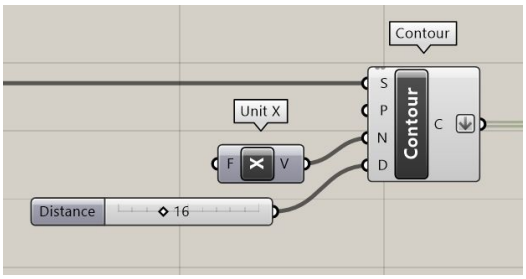
h) Then the model need to be imported in Rhino and a new Grasshopper environment needs to be created, inside of this new GH file we need to create the Block "Mesh", and link the imported STL file with our Grasshopper project.



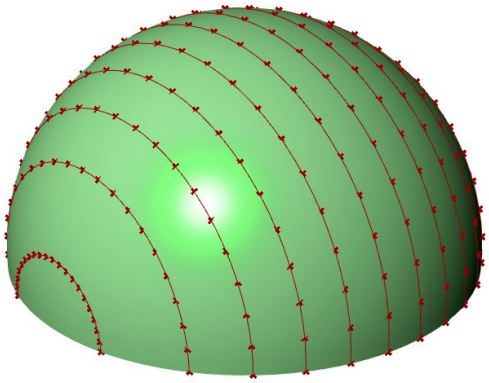


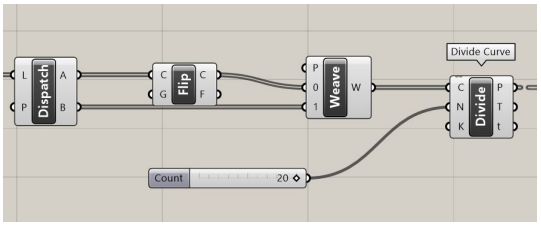
i) In order to slice our model, we need to generate the contour that would be created if we cut our model with a plane, to do this we insert the block "Contour", a "Unit X" to create a unit vector parallel to the world's X axis, and "Number slider" to configure the distance of each contour to generate.



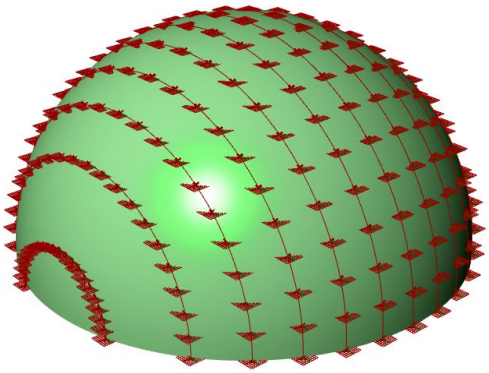


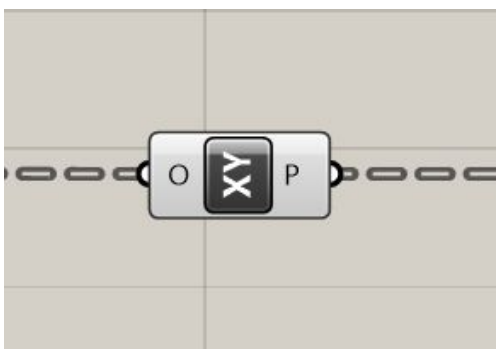
j) After we created the contours, we will use them to create our trajectories, we will split the curves and redirect them to generate a continuous path.



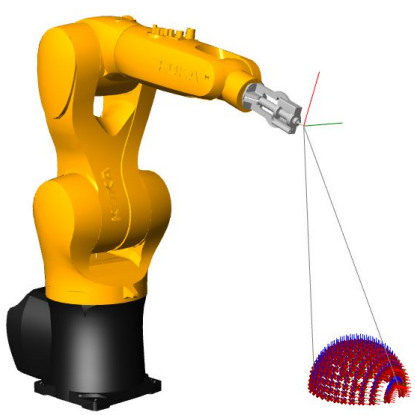


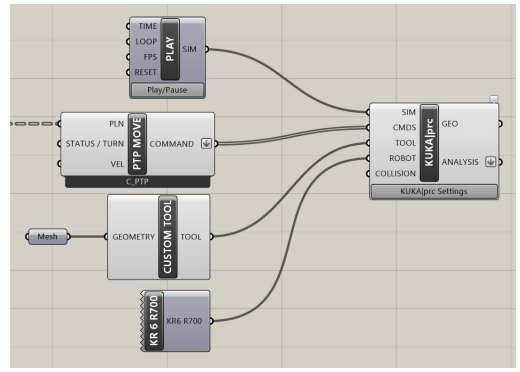
k) In each point created from the contours, we need to create planes to indicate the position in which our tool needs to be positioned when it passes.





l) And finally, we need to add the KUKA Prc modules to configure the robot, tool and the position of each one.

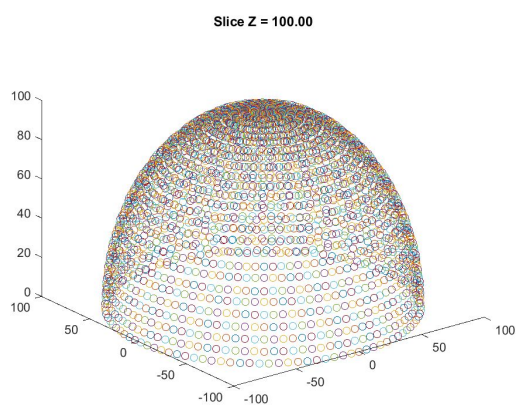




VI. Results

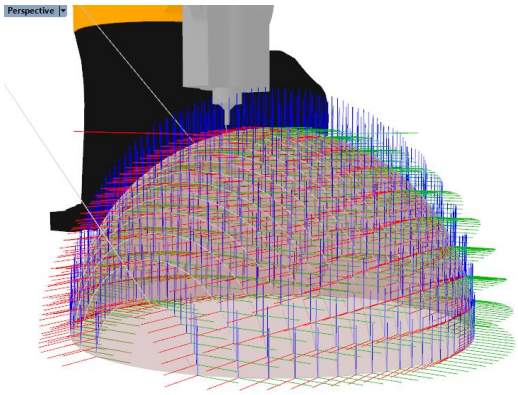
a) MATLAB method

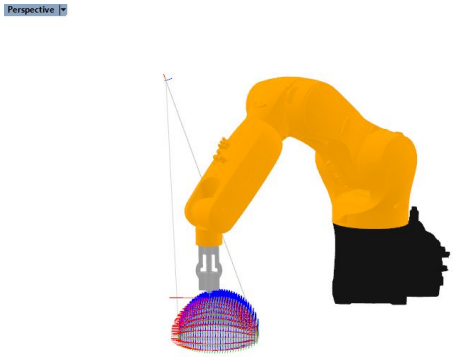
The results obtained in this method needs a lot of improvement, at this moment we only could get the points from the model, the second step is to connect these points with bezier curves or NURBS to get a smooth path. And finally would be the translation of the path into a Gcode for the robot.



b) Rhino method

The Rhino method gave the best results, as it was easier to implement, the downside is that it only works with KUKA robots, an improvement would be making the tool position normal to the trajectory. And translating the KUKA code to Gcode.





VII. Conclusion

This study shows approaches to path generation, Matlab and Rhino, with Matlab we work with the points cloud of the 3D model. the second is an approach with Rhino and Grasshopper. The Grasshopper method works in an easier way, as this is a parametric modeling with diagrams. The downside is that this method gives us the code to work with KUKA robots not with the robot proposed in this study.

Further investigation is needed, and improvements in the trajectories. Another improvement is the tool angle while following the path.

References

[1] K. Ma. (2020, May) Application of robotic-assisted *in situ* 3D printing in cartilage regeneration with HAMA hydrogel: An *in vivo* study, *Journal of Advanced Research*.. *23* , pp. 123–132. Available: https://doi.org/10.1016/j.jare.2020.01.010

[2] D. Chakraborty (2008, February) Extruder path generation for Curved Layer Fused Deposition Modeling, *ELSEVIER Computer-Aided Design. 24 (2),* pp 235-243. Available: https://doi.org/10.1016/j.cad.2007.10.014

[3] C. Wu, C. Dai, G. Fang, Y. Liu and C. C. L. Wang, "RoboFDM: A robotic system for support-free fabrication using FDM," 2017 IEEE International Conference on Robotics and Automation (ICRA), Singapore, 2017, pp. 1175-1180.

[4] J. Ogbemhe, K. Mpofu, N.S Tlale (2019) CONTINUOUS TRAJECTORY PLANNING FOR WELDING OF COMPLEX JOINTS USING BEZIER CURVE, *ELSEVIER Procedia Manufacturing, 23,* p 685-692.

[5] Westmoreland, J. (2017, May) How to Configure Universal Robots for Inverted Mounting. Retrieved from: https://www.crossco.com/blog/how-to-configure-universal-robots-for-inverted-mounting/